# XIAMEN UNIVERSITY MALAYSIA

| | | |
|---|---|---|
| Couse Code | : | MAT315 |
| Course Name | : | Time Series |
| Academic Session | : | September 2025 |
| Assessment | : | Project |

Prepared by :

| | | Report Mark | Presentation Mark |
|---|---|---|---|
| 1. Bong Xuan Jing | ID: DSC2304093 | | |
| 2. Sean Wong Wee Kai | ID: DSC2302025 | | |

**XIAMEN UNIVERSITY MALAYSIA**

## Own Work Declaration

I/We hereby understand my/our work would be checked for plagiarism or other misconduct, and the softcopy would be saved for future comparison(s).

I/We hereby confirm that all the references or sources of citations have been correctly listed or presented and I/we clearly understand the serious consequence caused by any intentional or unintentional misconduct.

This work is not made on any work of other students (past or present), and it has not been submitted to any other courses or institutions before.

Signature:

| Bong Xuan Jing | Sean Wong Wee Kai |
|---|---|
| DSC2304093 | DSC2302025 |

Date: 1/1/2025

## 1.0 Data Preprocessing

```
rice <- read.csv("C:/University/Year 3 Sem 2/Time Series/rice_yield_D.csv", stringsAsFactors = FALSE)

# Convert date column to Date class
rice$date <- as.Date(rice$date, format = "%d/%m/%Y")

start_year  <- as.numeric(format(min(rice$date), "%Y"))
start_month <- as.numeric(format(min(rice$date), "%m"))
start_year
start_month

rice.ts <- ts(rice$x, start = c(start_year, start_month), frequency = 12)
```

We majorly conduct 3 steps of data preprocessing.

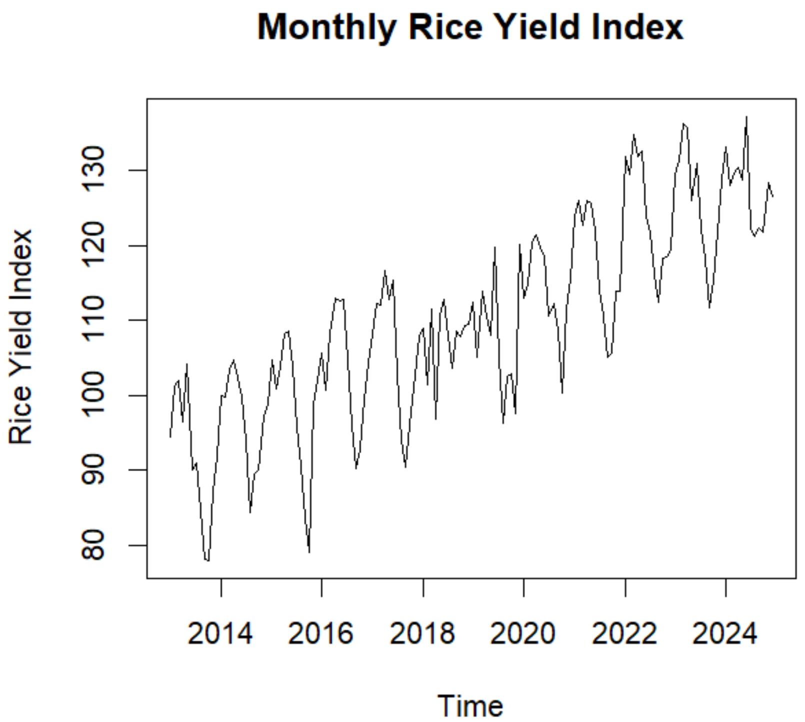
Step 1: Read in data & convert dates

Step 2: Extract the starting year & month

Step 3: Construct a monthly time series

## 2.0 Time Plot & Description

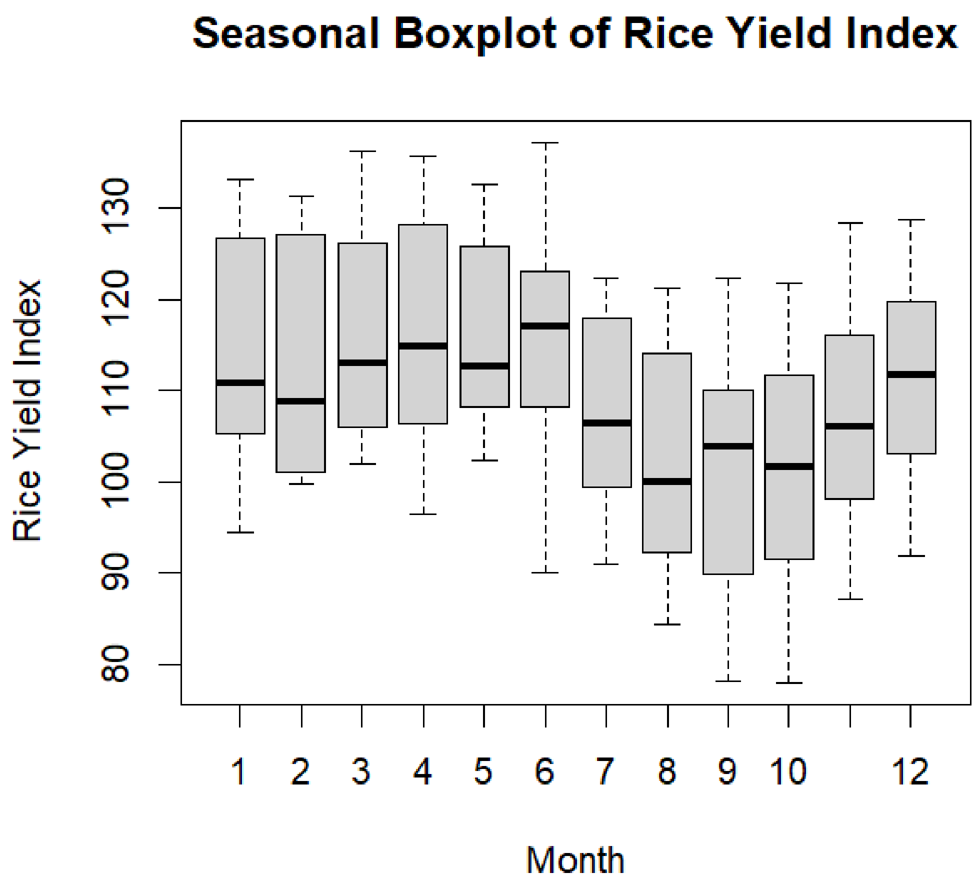
```
#1. Time Plot & Description
plot(rice.ts, main = "Monthly Rice Yield Index", ylab = "Rice Yield Index", xlab = "Time")
```




Trend: A clear increasing trend year by year (long - term upward trend)

Seasonality: There is also a clear seasonal cycle with periodic fluctuation (similar peaks and troughs occur each year), indicating the presence of an annual seasonal component.

```
boxplot(rice.ts ~ cycle(rice.ts), xlab = "Month", ylab = "Rice Yield Index",
        main = "Seasonal Boxplot of Rice Yield Index")
```

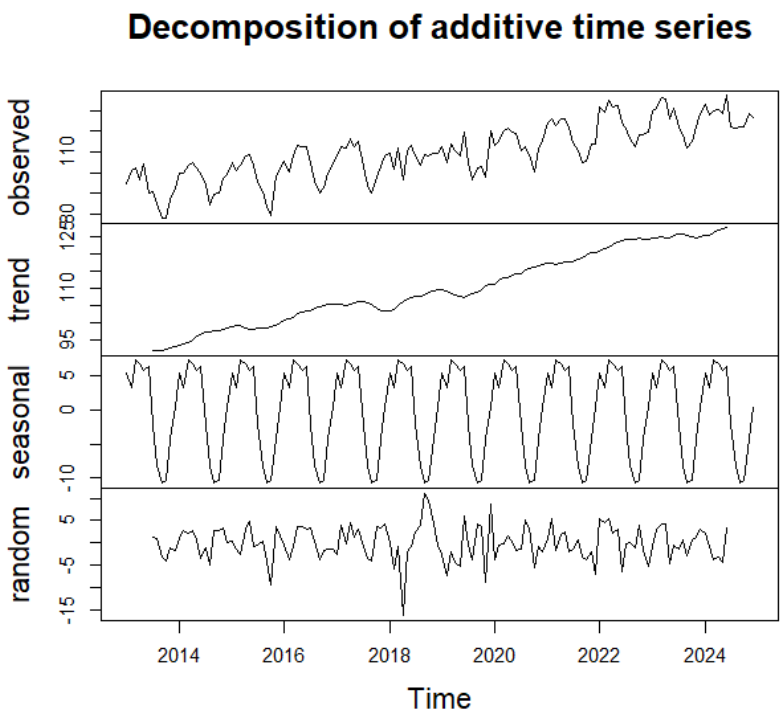


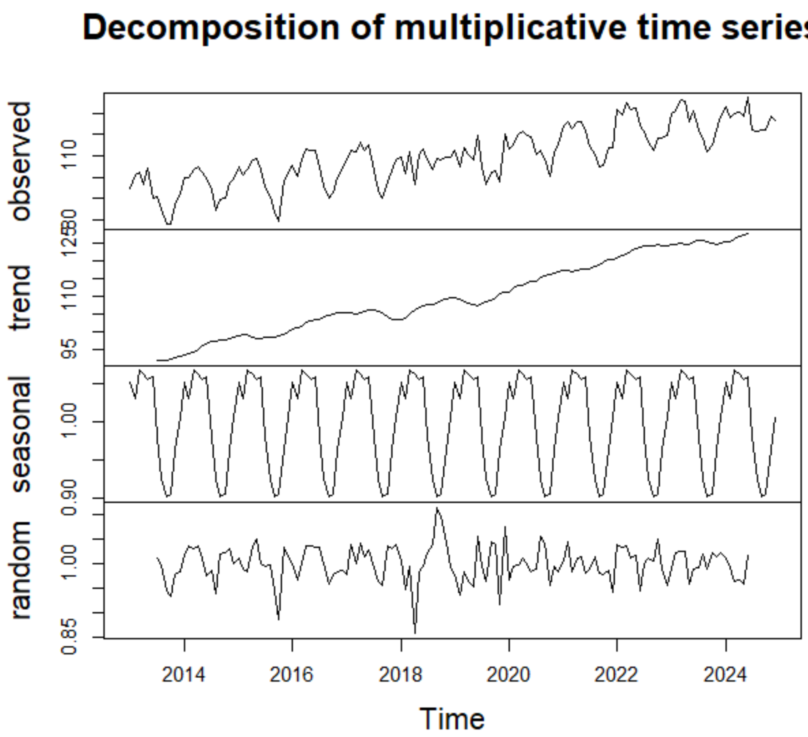

Seasonal Pattern: The median values of x were highest in June, and lowest in August, indicating that a seasonal pattern exists with a period of 12.

Seasonal Variation: The box heights vary across months (some months have greater dispersion/variation), indicating that seasonal fluctuations vary in degree across months.

## 3.0 Time Series Decomposition

```
#2. Time Series Decomposition
decomp_add <- decompose(rice.ts, type = "additive")
plot(decomp_add)

decomp_mul <- decompose(rice.ts, type = "multiplicative")
plot(decomp_mul)
```



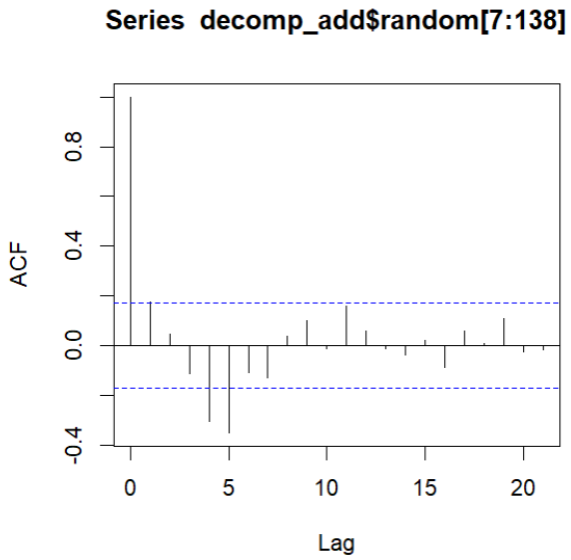



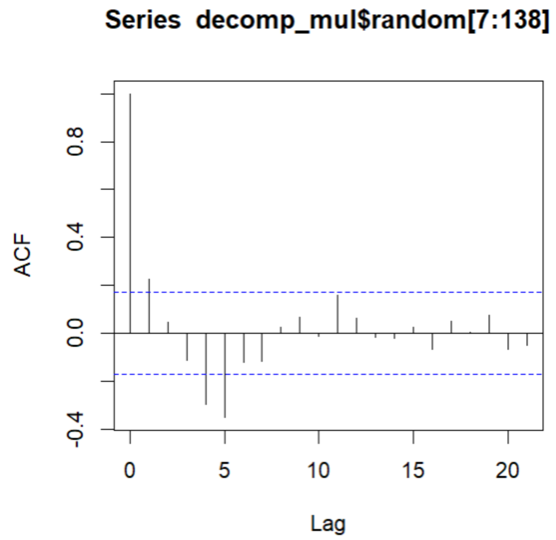

If the seasonal effect is constant as the trend increases, an additive decomposition model should be used. If the seasonal effect tends to increase as the trend increases, a multiplicative model is more appropriate.

Both additive and multiplicative decompositions were examined. Visually, the two decomposition plots appear very similar, indicating that the magnitude of the seasonal fluctuations does not vary substantially with the level of the series. Both models give similar results, therefore we select the simpler interpretation.

Given that the rice yield index is a standardized measure and the seasonal variation remains approximately constant over time, the additive decomposition is considered more appropriate for this dataset.

```
acf(decomp_add$random[7:138])
acf(decomp_mul$random[7:138])
```



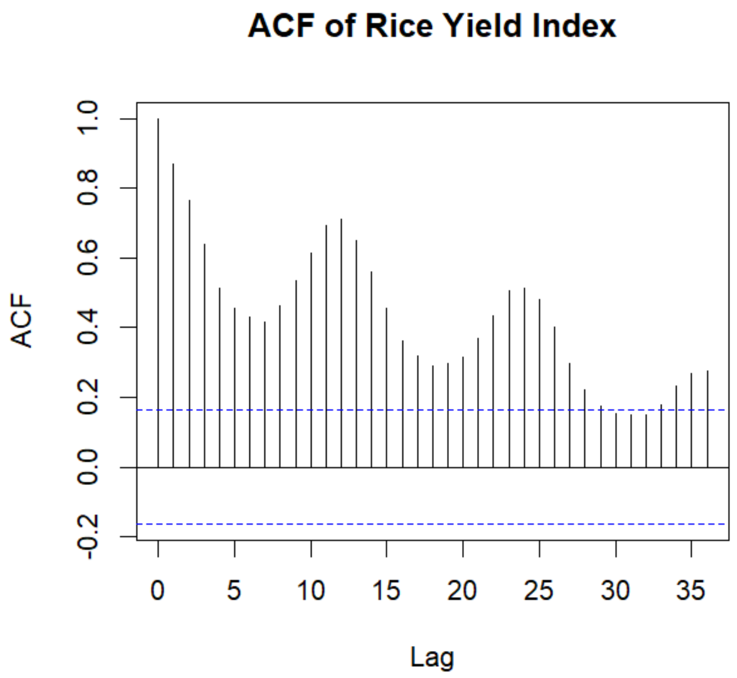




We also plot the correlogram of the random component of the additive and multiplicative decomposition model. We can observe that more spikes are statistically different from zero (especially lag 1) in the multiplicative model correlogram, indicating that the random component of the additive decomposition model is more likely to follow a white noise process, supporting the use of the additive model.

Besides, we can observe that the seasonal effect is almost constant as the trend increases from the time plot of original data, supporting the use of the additive model.

## 4.0 Correlation Structure

```
#3. Correlation Structure
acf(as.numeric(rice.ts), lag.max = 36, main = "ACF of Rice Yield Index")
```




The correlogram shows that the autocorrelation at small lags is very high and decays slowly as the lag increases. Even at large lags, the autocorrelations remain statistically significant. In addition, pronounced peaks are observed at lags 12 and 24, indicating strong annual seasonality.

These features are characteristic of a non-stationary time series with trend and seasonal components. Therefore, the rice yield index series appears to be non-stationary.

## 5.0 Fitting a Time Series Regression Model

We fit 4 time series regression models, as shown below:

1. An additive regression model with a linear trend and monthly seasonal indicators （OLS Method）

2. An additive regression model with a linear trend and monthly seasonal indicators （GLS Method）

3. Harmonic Seasonal Model

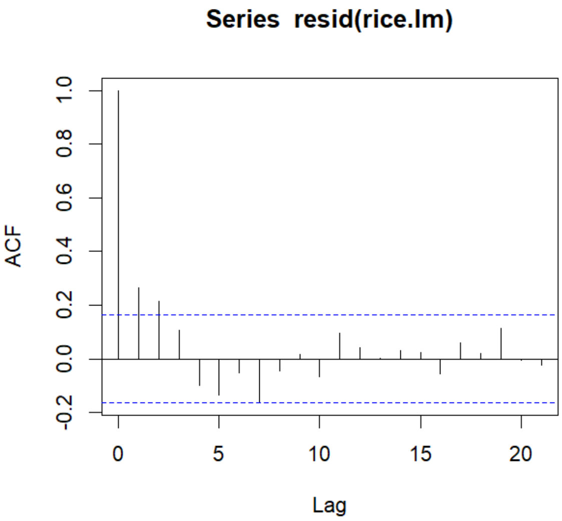
4. Reduced Harmonic Seasonal Model (retain only the significant sine and/or cosine terms)

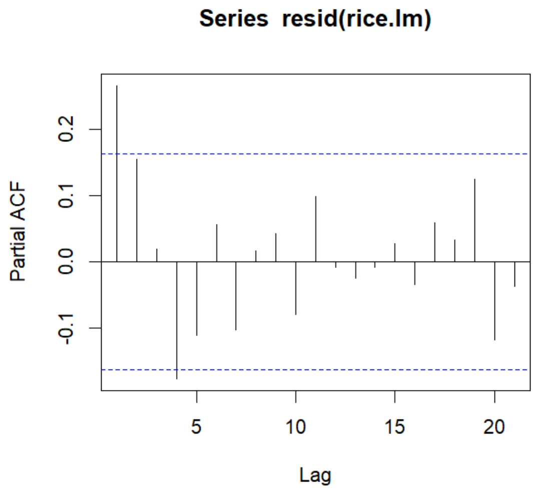
## 5.1 An additive regression model with a linear trend and monthly seasonal indicators (OLS Method)

By using the following code, we fit a time series regression model by using OLS method.

```
#4. Time Series Regression Model
#An additive regression model with a linear trend and monthly seasonal indicators
#(OLS Method)
Time   <- time(rice.ts)
Season <- cycle(rice.ts)

rice.lm <- lm(rice.ts ~ 0 + Time + factor(Season))
```

```
acf(resid(rice.lm))
pacf(resid(rice.lm))
```




The gradually decaying correlogram and a sharp cutoff partial correlogram at lag 4 suggest an AR(4) model for the residual series. Since there are autocorrelation appear within residual series, which means that OLS model is not suggested. We continue to fit a GLS model.

## 5.2 An additive regression model with a linear trend and monthly seasonal indicators (GLS Method)

By using the following code, we fit a time series regression model by using GLS method.

```
#(GLS Method)
library(nlme)
rice.gls <- gls(rice.ts ~ 0 + Time + factor(Season), cor = corARMA(p = 4))
```

## 5.3 Harmonic Seasonal Model

By using the following code, we fit a time series harmonic seasonal model.

The time standardization is necessary for reducing the computation error in the GLS procedure due to large numbers.

```r
#Harmonic Seasonal Model
SIN <- COS <- matrix(nr = length(rice.ts), nc = 6)
for (i in 1:6){
  COS[, i] <- cos(2 * pi * i * time(rice.ts))
  SIN[, i] <- sin(2 * pi * i * time(rice.ts))
}
meantime <- mean(time(rice.ts))
sdtime <- sd(time(rice.ts))
TIME <- (time(rice.ts) - meantime) / sdtime

rice.harmonic.gls <- gls(rice.ts ~ TIME + COS[, 1] + SIN[, 1]
                     + COS[, 2] + SIN[, 2] + COS[, 3] + SIN[, 3]
                     + COS[, 4] + SIN[, 4] + COS[, 5] + SIN[, 5]
                     + COS[, 6] + SIN[, 6], cor = corARMA(p = 4), method = "ML")
```

```
> coef(rice.harmonic.gls)/sqrt(diag(vcov(rice.harmonic.gls)))
(Intercept)        TIME    COS[, 1]    SIN[, 1]    COS[, 2]    SIN[, 2]    COS[, 3]    SIN[, 3]
232.5895673  23.6605156   3.6074029  10.7339081   2.5524908  -4.7297684   0.1009322   0.2484834
   COS[, 4]    SIN[, 4]    COS[, 5]    SIN[, 5]    COS[, 6]    SIN[, 6]
 -0.4629508  -1.5820593   2.0890590   0.4163398   0.7175680  -0.9566787
```

The harmonic coefficients are known to be independent, which means that all harmonic coefficients that are not statistically significant can be dropped. An approximate t-ratio of magnitude 2 is a common choice and corresponds to an approximate 5% significance level. This t-ratio can be obtained by dividing the estimated coefficient by the standard error of the estimate.

## 5.4 Reduced Harmonic Seasonal Model (retain only the significant sine and/or cosine terms)

```
> t_ratio <- coef(rice.harmonic.gls)/sqrt(diag(vcov(rice.harmonic.gls)))
> t_ratio[t_ratio > 2 | t_ratio < -2]
(Intercept)        TIME    COS[, 1]    SIN[, 1]    COS[, 2]    SIN[, 2]    COS[, 5]
 232.589567   23.660516    3.607403   10.733908    2.552491   -4.729768    2.089059
```

The sine and cosine terms that are statistically significant including

COS[, 1] , SIN[, 1] , COS[, 2] , SIN[, 2] , COS[, 5], so we fit another model with retain only the significant sine and/or cosine terms.

```r
#Harmonic Seasonal Model with only retain the significant sine and / or cosine terms
rice.harmonic.gls.reduced <- gls(rice.ts ~ TIME + COS[, 1] + SIN[, 1] + COS[, 2] + SIN[, 2] + COS[, 5],
                     cor = corARMA(p = 4), method = "ML")
```

## 5.5 Find the best regression model

```
> #Compare 4 Regression Model
> AIC(rice.lm)
[1] 852.4451
> AIC(rice.gls)
[1] 817.865
> AIC(rice.harmonic.gls)
[1] 842.6741
> AIC(rice.harmonic.gls.reduced)
[1] 834.4149
```

Four regression-based models were fitted and compared using the Akaike Information Criterion (AIC). The additive regression model with a linear trend and monthly seasonal indicators estimated using GLS (The second model) achieved the lowest AIC value, indicating the best balance between model fit and complexity.

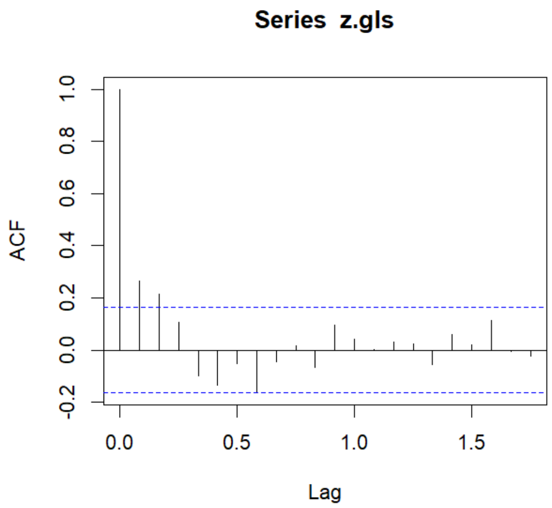
Although harmonic seasonal models provide a parsimonious representation of seasonality, they assume a smooth sinusoidal pattern. In this dataset, the seasonal effects appear to be irregular across months, which is better captured by monthly seasonal indicators. As a result, the harmonic and reduced harmonic models yield higher AIC values and do not outperform the additive seasonal regression model.

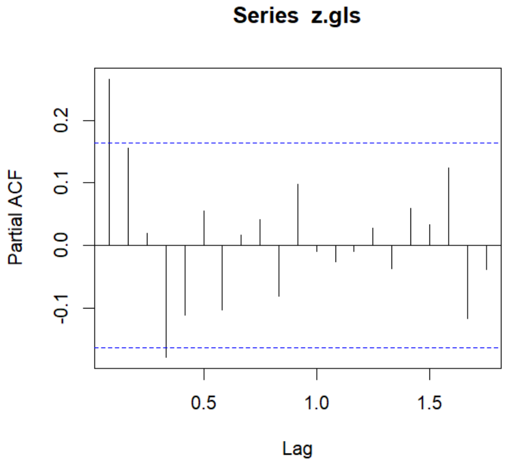
The advantage of the harmonic model is that it approximates seasonality with a small number of parameters. However, if there is a large amount of data or the monthly seasonality is well-defined, the harmonic model is not suitable.

## 5.6 Further Details Analysis of the best model

```
> #Detail Analysis of Second Model
> summary(rice.gls)
Generalized least squares fit by REML
  Model: rice.ts ~ 0 + Time + factor(Season)
  Data: NULL
      AIC      BIC    logLik
  817.865 869.6186 -390.9325

Correlation Structure: ARMA(4,0)
 Formula: ~1
 Parameter estimate(s):
      Phi1       Phi2       Phi3       Phi4
 0.2292698  0.1864448  0.0628948 -0.1775574

Coefficients:
                     Value Std.Error   t-value p-value
Time                 3.241   0.14318  22.63688       0
factor(Season)1  -6428.631 289.03954 -22.24136       0
factor(Season)2  -6430.177 289.04037 -22.24664       0
factor(Season)3  -6426.447 289.04086 -22.23370       0
factor(Season)4  -6427.236 289.04564 -22.23606       0
factor(Season)5  -6427.495 289.06169 -22.23572       0
factor(Season)6  -6428.118 289.07654 -22.23673       0
factor(Season)7  -6436.343 289.09324 -22.26390       0
factor(Season)8  -6441.967 289.10808 -22.28221       0
factor(Season)9  -6444.280 289.12413 -22.28898       0
factor(Season)10 -6444.092 289.12891 -22.28796       0
factor(Season)11 -6437.959 289.12938 -22.26671       0
factor(Season)12 -6433.699 289.13019 -22.25191       0
```

```
z.gls <- resid(rice.gls)
acf(z.gls)
pacf(z.gls)
```




The gradually decaying correlogram and a sharp cutoff partial correlogram at lag 4 suggest an AR(4) model for the residual series.

We fit the residual to a AR(4) model.

```
> z.gls.ar <- ar(z.gls, method = "mle")
> z.gls.ar

Call:
ar(x = z.gls, method = "mle")

Coefficients:
      1        2        3        4
 0.2225   0.1814   0.0592  -0.1833

Order selected 4  sigma^2 estimated as  15.72
> z.gls.ar$ar
[1]  0.22254554  0.18144993  0.05917095 -0.18334302
> mean(z.gls)
[1] 0.0181724
```

Fitted equation of residual:

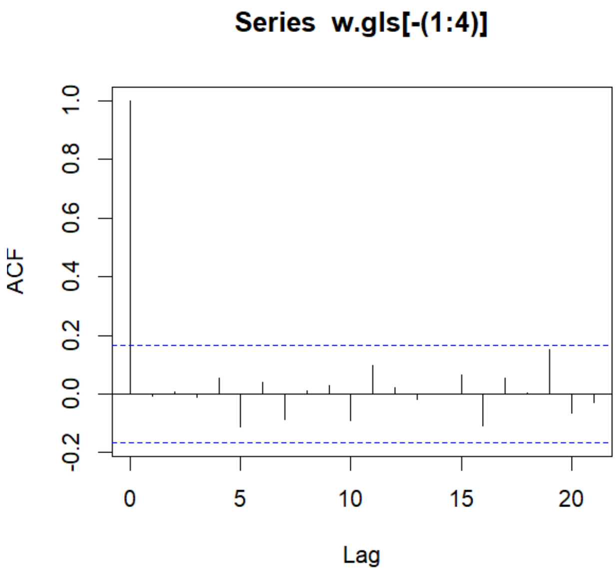
$$z_t - \mu = \alpha_1(z_{t-1} - \mu) + \alpha_2(z_{t-2} - \mu) + \alpha_3(z_{t-3} - \mu) + \alpha_4(z_{t-4} - \mu) + w_t$$

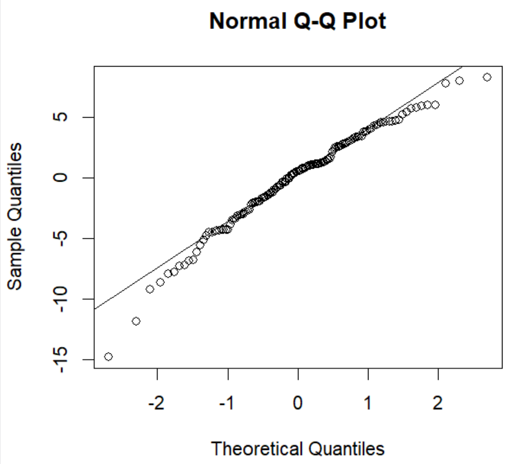
$$z_t - 0.0181724 = 0.22254554(z_{t-1} - 0.0181724) + 0.18144993(z_{t-2} - 0.0181724) + 0.05917095(z_{t-3} - 0.0181724) - 0.18334302(z_{t-4} - 0.0181724) + w_t$$

$$z_t = 0.0131 + 0.2225z_{t-1} + 0.1814z_{t-2} + 0.0592z_{t-3} - 0.1833z_{t-4} + w_t$$

We check whether $w_t$ is a white noise:

```
w.gls <- z.gls.ar$resid
acf(w.gls[-(1:4)])
qqnorm(w.gls[-(1:4)])
qqline(w.gls[-(1:4)])
```

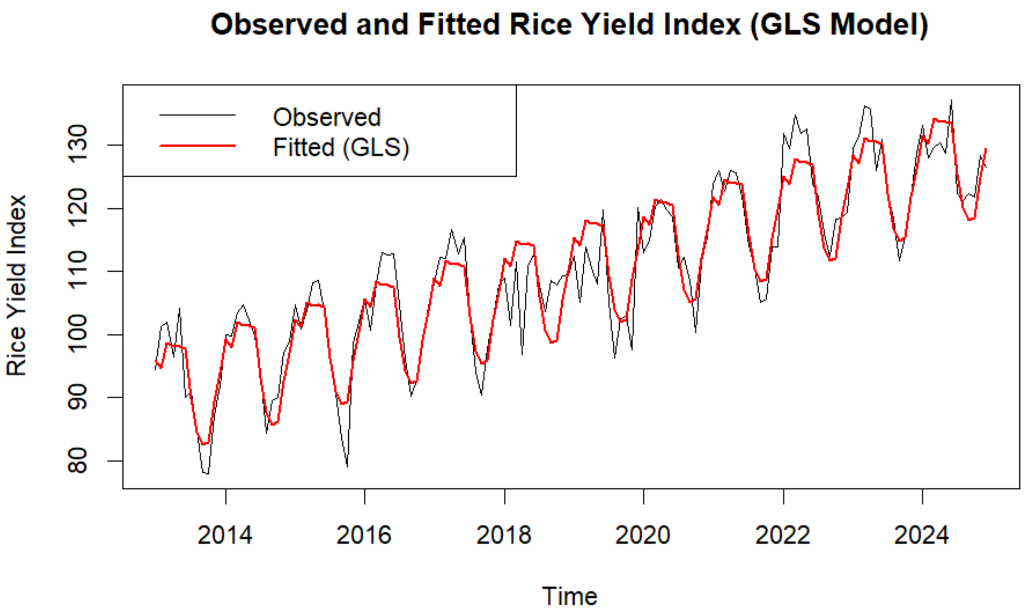



The quick decaying correlogram and the close fit of the normal qq line to the point suggest that $w_t$ is a Gaussian white noise with mean 0 and $\sigma^2 = 15.72$.

We compare the fitted and observed values by using the following code.

```
# Fitted values from GLS model
fitted.gls <- fitted(rice.gls)

fitted.gls.ts <- ts(fitted.gls,
                    start = start(rice.ts),
                    frequency = frequency(rice.ts))
plot(rice.ts,
     main = "Observed and Fitted Rice Yield Index (GLS Model)",
     ylab = "Rice Yield Index",
     xlab = "Time")

lines(fitted.gls.ts, col = "red", lwd = 2)

legend("topleft",
       legend = c("Observed", "Fitted (GLS)"),
       col = c("black", "red"),
       lty = c(1,1),
       lwd = c(1,2))
```

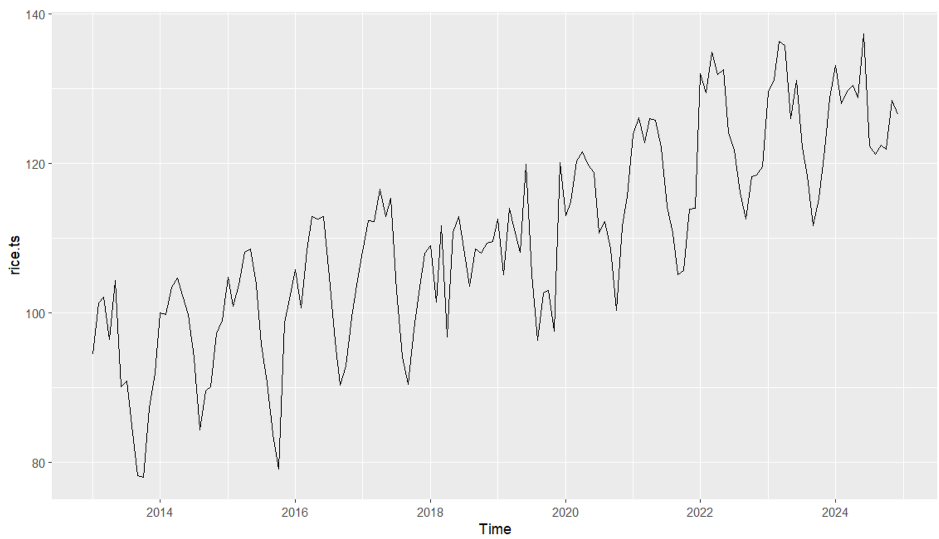



The model shows a good fit to the data since the model captures most of the features and is close to the original data.

## 6.0 Fitting a model from among AR, MA, ARMA, ARIMA, or SARIMA

## 6.1 Determining seasonality and trend

The rice yield is plotted to check for seasonality and trend.



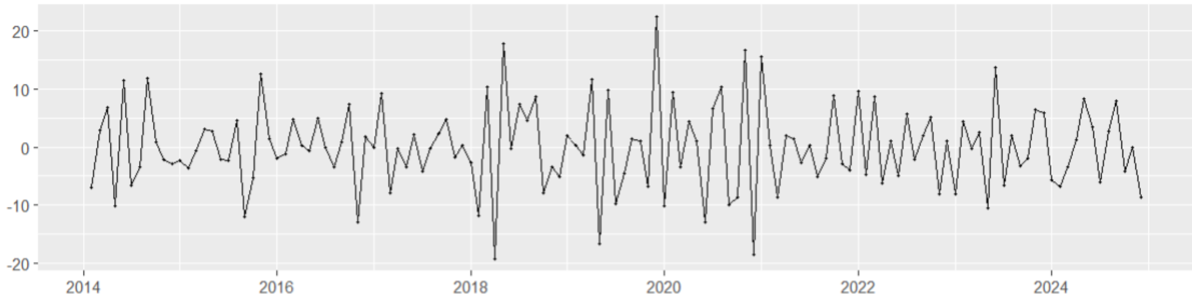

There is clear seasonality and trend in the rice yield time-series plot (this was done in part 2.0). Standard AR, MA, ARMA models require data to be stationary (no trend). A standard ARIMA model only considers consecutive observations (no seasonality). Therefore, SARIMA should be chosen to fit this time-series because it factors in both trend and seasonality.

## 6.2 Determining non-seasonal and seasonal difference

ndiffs and nsdiffs are functions that automatically determine how much differencing is needed to make the time series stationary.

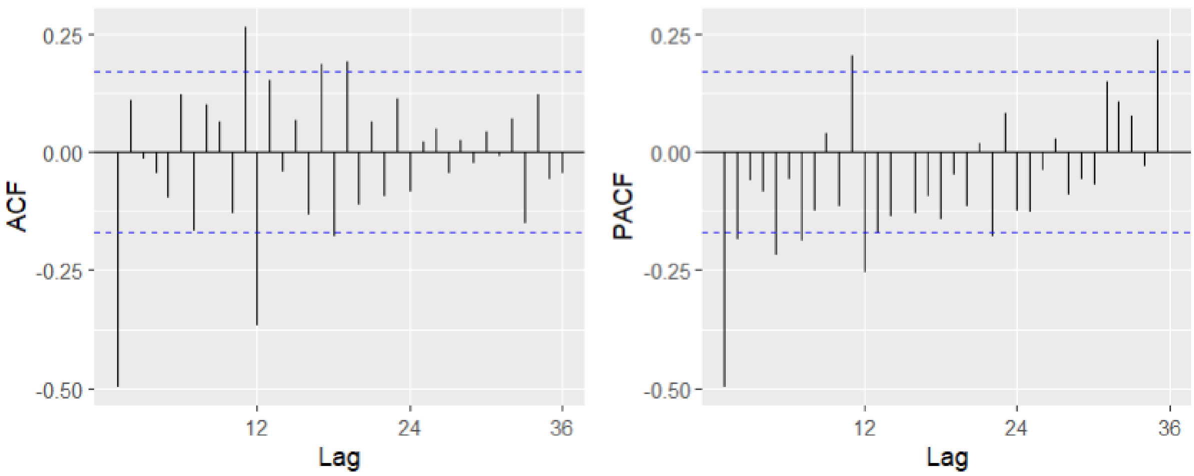
```
> ndiffs(rice.ts)
[1] 1
> nsdiffs(rice.ts)
[1] 1
```

The results show that non-seasonal difference, $d = 1$ and seasonal difference, $D = 1$.




It appears that the trend and seasonality is eliminated in the plot of residuals after differencing.

## 6.3 Determining non-seasonal and seasonal autoregressive and moving average terms




From both the ACF and PACF plot of the differenced time series, there is a significant spike at lag 12, which justifies the need of setting the parameters P and Q to be at least 1.

```r
get.best.sarima <- function(x.ts, maxord = c(1, 1, 1, 1, 1, 1)) {
  best.aic <- Inf
  n <- length(x.ts)

  for (p in 0:maxord[1]) for (d in 0:maxord[2]) for (q in 0:maxord[3])
    for (P in 0:maxord[4]) for (D in 0:maxord[5]) for (Q in 0:maxord[6])
    {
      fit <- arima(x.ts, order = c(p, d, q),
                   seasonal = list(order = c(P, D, Q),
                                   frequency(x.ts)), method = "CSS")

      fit.aic <- -2 * fit$loglik + (log(n) + 1) * length(fit$coef)

      if (fit.aic < best.aic) {
        best.aic <- fit.aic
        best.fit <- fit
        best.order <- c(p, d, q, P, D, Q)
      }
    }

  return(list(best.aic = best.aic, best.fit = best.fit, best.order = best.order))
}

rice.best <- get.best.sarima(rice.ts, maxord = c(3,1,3,3,1,3))
rice.best
```

Since non-seasonal and seasonal difference are both set to 1, the best-fitting model is searched using terms up to the third order apart from those two parameters. A grid search function is used for this. It uses nested loops to iterate all possible combinations and fits them into a model. The best fit is then updated by calculating the fit.aic, which is $-2 \times$ log-likelihood + $(\log(n) + 1) \times$ number of parameters. This evaluation metric factors in number of parameters to avoid parametrisation.

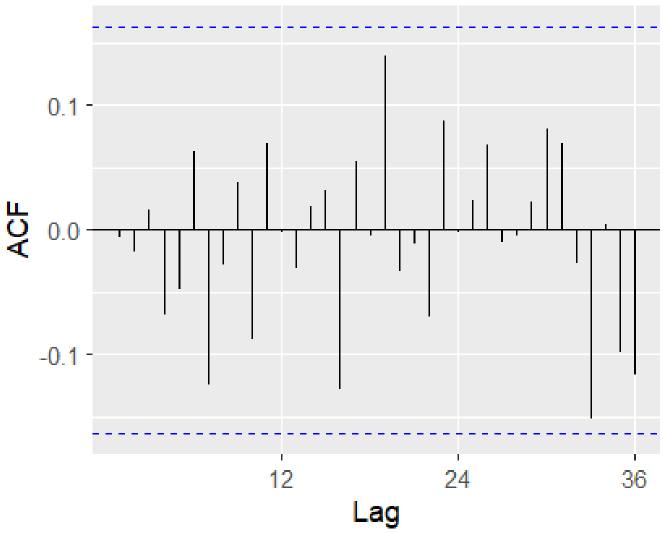
```
> rice.best
$best.aic
[1] 799.1503

$best.fit

Call:
arima(x = x.ts, order = c(p, d, q), seasonal = list(order = c(P, D, Q), frequency(x.ts)),
    method = "CSS")

Coefficients:
         ar1      ar2      ma1     ma2      ma3     sar1     sma1     sma2
      0.6609  -0.3619  -1.5005  0.9601  -0.5380  -0.3435  -0.4750  -0.4244
s.e.  0.0011      NaN   0.0480  0.1299   0.0766      NaN   0.0764   0.0788

sigma^2 estimated as 18.14:  part log likelihood = -375.7

$best.order
[1] 2 1 3 1 1 2
```

The SARIMA model with best fit is $(2,1,3)(1,1,2)_{12}$.

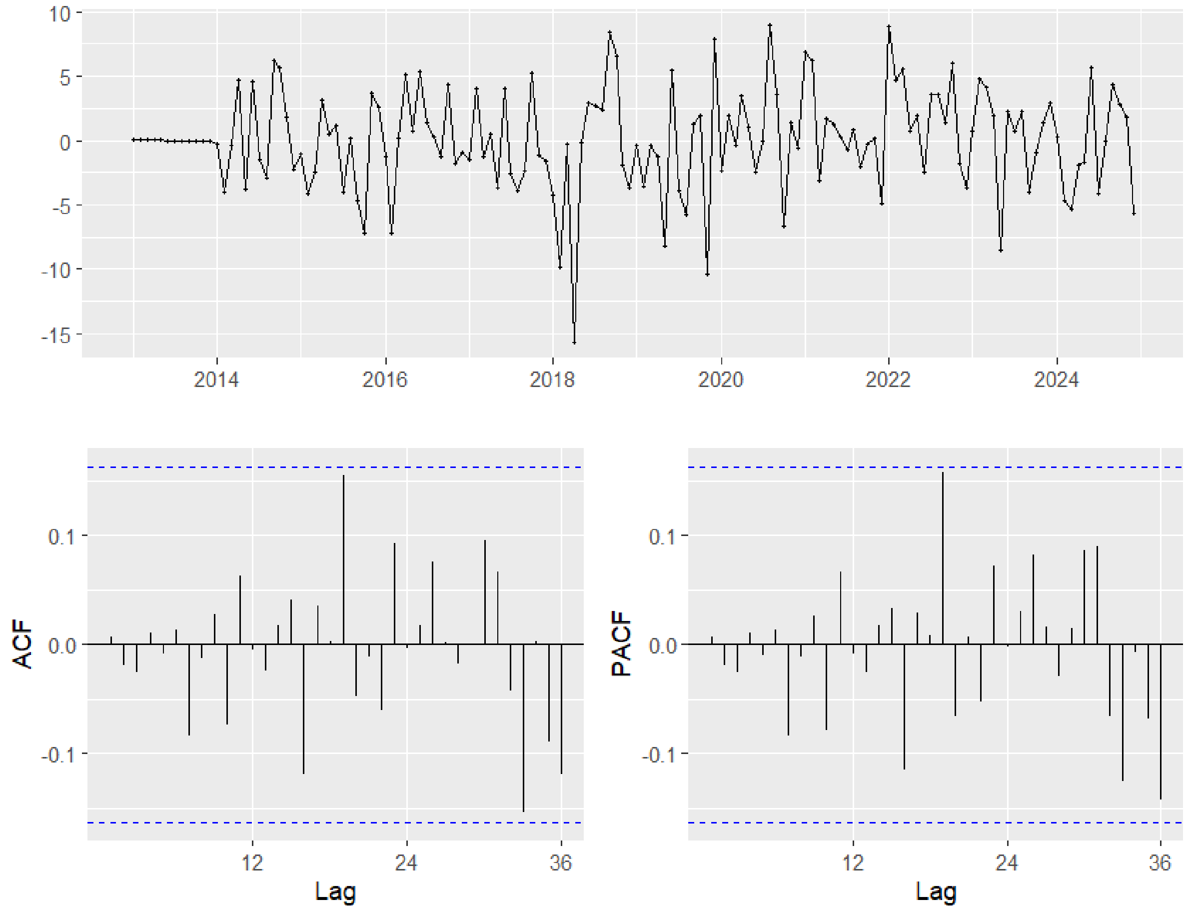
## 6.4 Verifying the model

```
        Ljung-Box test

data:  Residuals from ARIMA(2,1,3)(1,1,2)[12]
Q* = 15.568, df = 16, p-value = 0.4835

Model df: 8.   Total lags used: 24
```

The null hypothesis for the Ljung-Box test is that the residuals are independently distributed and there is no significant autocorrelation in the data. Since the p-value is greater than 0.05, we fail to reject the null hypothesis. Therefore, this confirms that the model is a good fit.




Although higher-order terms could be tried but that would seem unnecessary since the residuals appear to be white noise. But for the sake of comparison, we will fit a SARIMA model with parameters $(6,1,1)(1,1,1)_{12}$.



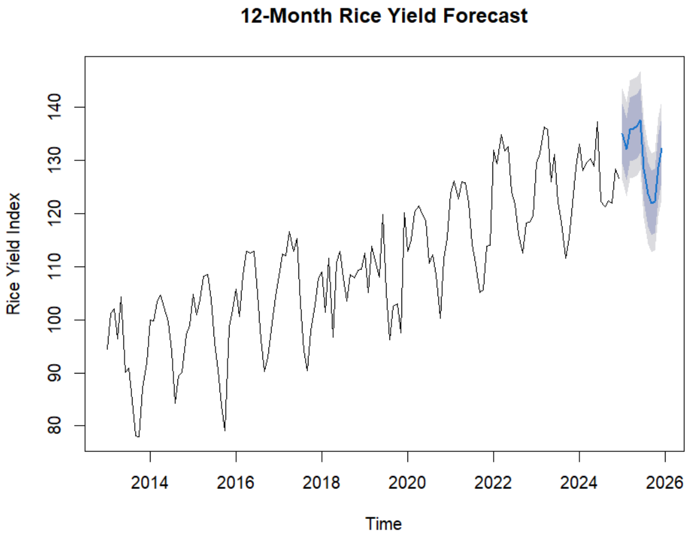

```
Ljung-Box test

data:  Residuals from ARIMA(6,1,1)(1,1,1)[12]
Q* = 12.433, df = 15, p-value = 0.646

Model df: 9.   Total lags used: 24
```

The new model fitted appears to be a good fit too. However, there is one less degree of freedom compared to the original model. We will show another reason why we prefer the original model in the following image.

```
> n <- length(model$residuals)
> fit.aic <- -2 * model$loglik + (log(n) + 1) * length(model$coef)
> fit.aic
[1] 827.9621
```

It can be seen that its consistent Akaike Information Criteria (CAIC) of the new model is 827.9621, which is greater than the original model's CAIC of 799.1503 (refer to part 6.3), therefore we should retain the original model.

## 7.0 Forecasting



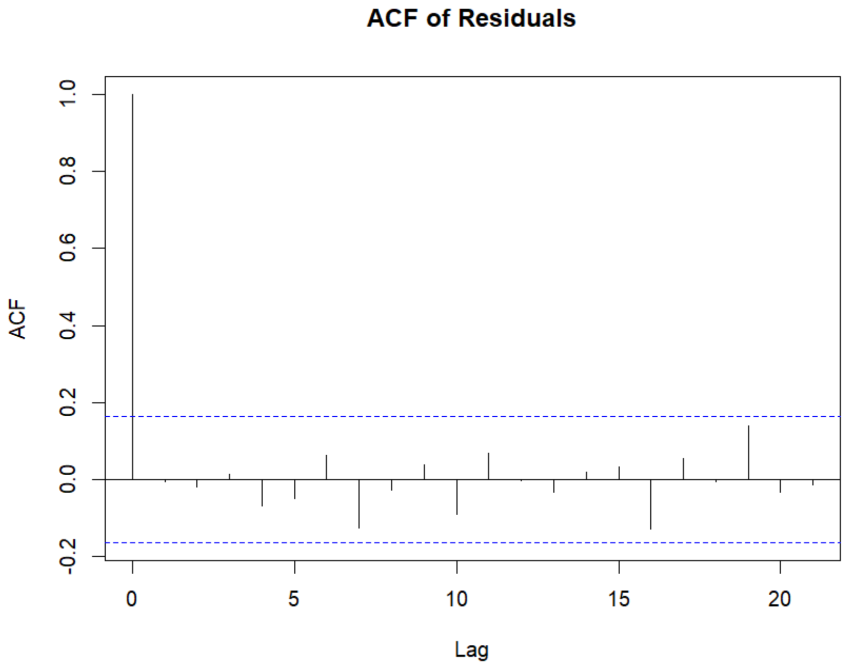

Forecasts:

| | Point Forecast | Lo 80 | Hi 80 | Lo 95 | Hi 95 |
|---|---|---|---|---|---|
| Jan 2025 | 134.9758 | 129.3310 | 140.6207 | 126.3428 | 143.6089 |
| Feb 2025 | 132.1330 | 126.3413 | 137.9246 | 123.2753 | 140.9906 |
| Mar 2025 | 135.8721 | 129.8668 | 141.8774 | 126.6878 | 145.0565 |
| Apr 2025 | 136.0422 | 129.9603 | 142.1242 | 126.7407 | 145.3437 |
| May 2025 | 136.4655 | 130.3800 | 142.5510 | 127.1586 | 145.7725 |
| Jun 2025 | 137.5464 | 131.4631 | 143.6298 | 128.2428 | 146.8501 |
| Jul 2025 | 128.4690 | 122.3877 | 134.5502 | 119.1685 | 137.7694 |
| Aug 2025 | 123.6920 | 117.6139 | 129.7701 | 114.3964 | 132.9877 |
| Sep 2025 | 121.9825 | 115.9043 | 128.0606 | 112.6867 | 131.2782 |
| Oct 2025 | 122.2994 | 116.2176 | 128.3812 | 112.9981 | 131.6008 |
| Nov 2025 | 128.7490 | 122.6629 | 134.8351 | 119.4412 | 138.0568 |
| Dec 2025 | 132.2321 | 126.1433 | 138.3209 | 122.9201 | 141.5441 |

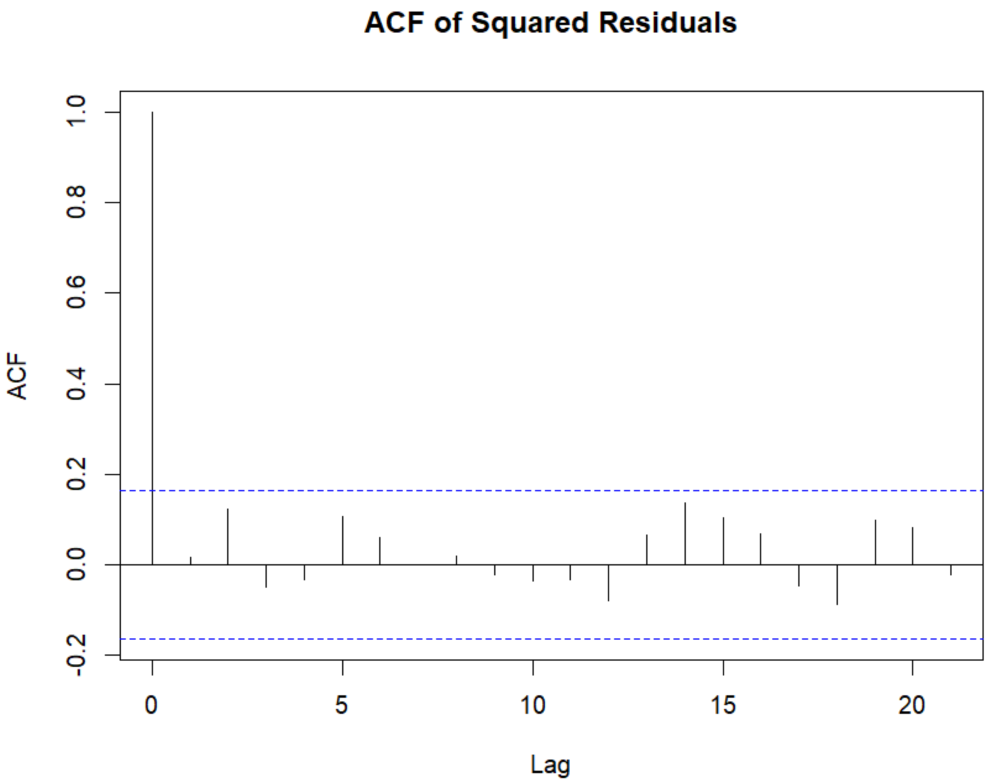
After using the original model for forecasting the next 12 months with 95% confidence interval (lighter shaded grey region), we can see that the rice yield is expected to reach its highest point for the year in June 2025 with a point forecast of 137.55. There will be a significant decline starting after June, reaching a lowest point in September 2025 (121.98). Starting from October, the index is forecasted to begin a recovery back towards 132.23 by December 2025.

## 8.0 Volatility Check

Volatility clustering occurs when periods of high volatility are followed by high volatility, and periods of low volatility are followed by low volatility. This is checked using the ACF plot of squared residuals after confirming that there are no significant spikes in the ACF plot of residuals.




This was confirmed in part 6.4.




In the ACF plot for squared residuals, all spikes after lag 0 remain within the blue dashed boundaries that represent the 95% confidence interval. This suggests that there is no significant autocorrelation in the squared errors. Hence, it is not necessary to include an ARCH or GARCH model.

## Contributions

All group members had contributed actively to the project. Regular discussions were conducted to review progress, solve issues collaboratively, and maintain consistency in both the coding work and the report.

| Name | Task |
|---|---|
| Bong Xuan Jing<br>DSC2304093 | Time Plot & Description,<br>Time Series Decomposition,<br>Correlation Structure,<br>Fitting a Time Series Regression Model |
| Sean Wong Wee Kai<br>DSC2302025 | Fit and Compare Models,<br>Forecasting,<br>Volatility Checking |